

DNA 计算, 生物数学

限制性内切酶

学科与专题介绍

③

DNA 计算: 生物数学的来临 (续)

282-291, 273

LiLa Kari

Kari, LL Q523

Q-332

一个数学模型: 剪接系统

由于 DNA 链可以比作在四字符集上的串, 为 DNA 计算建模的自然方式就是利用专门处理字符和字符串的形式语言理论. 在描述剪接系统的技术背景和形式定义之前, 我们简要的描述一下怎样将实际的 DNA 重组抽象为数学上的剪接操作.

考虑下面两个双链 DNA:

$$5'CCCCCTCGACCCCC3' \quad 5'AAAAAGCGCAAAAA3'$$

$$3'GGGGGAGCTGGGGG5' \quad 3'TTTTTTCGCGTTTTT5'$$

和两个限制性内切酶 (*TaqI* 和 *SciNI*), 其识别位点分别是:

$$5'TCGA3' \quad 5'GCGC3'$$

$$3'AGCT5' \quad 3'CGCG5'$$

酶在这两个 DNA 链上的作用是, 各个酶将包含其限制性位点的链切割成子序列. 结果是生成四个新的 DNA 链:

$$5'CCCCCT \quad CGACCCCC3' \quad 5'AAAAAG \quad CGCAAAAA3'$$

$$3'GGGGGAGC \quad TGGGGG5' \quad 3'TTTTTTCGC \quad GTTTTT5'$$

注意第一个链的粘性末端与最后一个链的粘性末端是互补的, 第二和第三个链也是如此. 带有相容末端的 DNA 链可以由 DNA 连接酶连接在一起, 结果就成了:

$$5'CCCCCTCGCAAAAA3' \quad 5'AAAAAGCGACCCCCA3'$$

$$3'GGGGGAGCGTTTTT5' \quad 3'TTTTTTCGCTGGGGG5'$$

原题: DNA Computing: Arrival of Biological Mathematics. 译自: The Mathematical Intelligencer 19(1997), No. 2, 9-22.

上述由酶执行的切割和粘贴的组合过程是 DNA 重组的一个例子. 其数学抽象称为剪接操作, 该操作定义在如下的假设上:

(i) 我们要考虑的是符号串, 而不是双链结构 (由于 A-T, C-G 的互补性, 这样作并不会丢失什么信息)

(ii) 字符集的大小并不限于四个字符 (该推广应当是合理的, 因为任何字符集都可以用二进制表示).

(iii) 位点的长度及同时起作用的酶的数量不受限制. (这是一个非常宽松的假设, 但剪接系统的目标是建立 DNA 计算的抽象通用模型, 而不必受限于目前的生物技术.)

有了这此假设, 我们几乎就可以将剪接操作剪接系统定义为数学对象了, 但我们还需要几个数学定义和符号.

字符集是一个非空有限集, 其元素称为字符或符号. Σ^* 表示在字符集 Σ 上由连接操作生成的字符串的集合. Σ^* 中的元素称为词或串. 空串 (Σ^* 中的空元素) 记作 λ . 字符集 Σ 上的语言是 Σ^* 的一个子集. 例如, 若 $\Sigma = \{a, b\}$, 那么 $aaba, aabbb = a^3b^2$ 是 Σ 上的词, 下述的几个集合都是 Σ 上的语言: $L_1 = \{\lambda\}$, $L_2 = \{a, ba, aba, abba\}$, $L_3 = \{a^p | p \text{ 是素数}\}$.

由于语言定义为集合, 我们就可以在语言上利用集合论中的并, 交, 差, 补等操作. 如语言 L_1, L_2 的联结操作, 记为 $L_1 L_2$, 定义为 $L_1 L_2 = \{uv | u \in L_1, v \in L_2\}$.

定义有限语言的方法是列出其中所有的词, 但对于无限语言, 这种方法就不适用了. 因此人们开发了几种定义无限语言的方法, 其中之一是定义一个生成器, 然后将语言定义为由生成器所生成的所有词的集合.

生成语法是一个有序四元组:

$$G = (N, T, S, P)$$

其中 N, T 是互不相交的字符集, $S \in N, P$ 是由有序对 (u, v) 构成的有限集, 其中 u, v 是 $N \cup T$ 上的词, 而且 v 至少包含一个 N 中的字符. N 元素称为非终结符, T 元素称为终结符, S 称为公理. P 的元素 (u, v) 称为重写规则或产生式, 记作 $u \rightarrow v$. 若 $x = x_1 u x_2, y = y_1 v y_2, u \rightarrow v \in P$, 那么可记为 $x \Rightarrow y$, 我们称在语法 G 中可以由 x 得到 y . 生成规则 $\Rightarrow$ 的自反传递闭包记为 $\Rightarrow^*$, 由 G 生成的语言为

$$L(G) = \{w \in T^* | S \Rightarrow^* w\}$$

直观的说, 语法 G 生成的语言是终结字符集上的词的集合, 这些词是由从公理开始不断地利用重写规则得到的.

语法可以按在产生式格式上的不同限制分类. 如果格式上没有任何限制, 称作 0 型语法; 如果格式形如: $A \rightarrow aB, A \rightarrow a$, 则称作正则语法, 这里 $A, B \in N, a \in TV \setminus \{\lambda\}$. 有限语言的集合记作 FIN, 由正则语法生成的语言集合称为 REG, 由 0 型语言生成的语言集合记作 $\mathcal{L}$.

有了这些形式语言理论的预备知识, 我们就可以定义剪接操作了.

[22] 中给出了剪接操作的定义并在 [18] 中得到修正. 给定字符集 Σ 及其上的两个串 x 和 y , 利用剪接规则 r 剪接 x 和 y 的过程可以分为两步: (i) 在由剪接规则 r 决定的位置上切割 x 和 y ; (ii) 分别将结果中 x 的前段和 y 的后段, y 的前段和 x 的后段连在一起. 利用 [37] 中的形式化方法, Σ 上的剪接规则 r 是形如 $\alpha_1\#\beta_1\$ \alpha_2\#\beta_2$ 的词, 其中 $\alpha_1, \beta_1, \alpha_2, \beta_2$ 是 Σ 上的串, $\#$ 和 $\$$ 是 Σ 外的标记符.

我们称 z 和 w 是根据剪接规则 $r = \alpha_1\#\beta_1\$ \alpha_2\#\beta_2$ 剪接 x 和 y 的结果, 当且仅当存在 $x_1, x'_1, y_1, y'_1 \in \Sigma^*$, 使得

$$x = x_1\alpha_1\beta_1x'_1$$

$$y = y_2\alpha_2\beta_2y'_2$$

且

$$z = x_1\alpha_1\beta_2y'_2$$

$$w = y_2\alpha_2\beta_1x'_1$$

并记作 $(x, y) \rightarrow_r (z, w)$.

$\alpha_1\beta_1$ 和 $\alpha_2\beta_2$ 这两个词称为剪接位点; x 和 y 称为剪接项. 剪接规则 r 决定了切割的位点和位置: 第一项在 α_1 和 β_1 之间, 第二项在 α_2 和 β_2 之间. 值得注意的是位点 $\alpha_1\beta_1$ 和 $\alpha_2\beta_2$ 会分别在 x 和 y 中出现多次, 如果这样, 选择哪一个位点是不确定的. 结果会造成对 x 和 y 剪接的结果是 (z, w) 的一个集合.

下面演示一下用剪接来模拟将两个正整数 m 和 n 相加的方法. 考虑字符集 $\Sigma = \{a, b, c\}$, 剪接规则 $r = a\#b\$c\#a$, 然后根据 r 对 $x = a^n b$ 和 $y = c a^m$ 进行剪接产生 a^{m+n} 和 cb 两个词:

$$\begin{aligned} (a^n b, C a^m) &= (x, y) \\ &= (\underbrace{a^{n-1}}_{x_1} \underbrace{a}_{\alpha_1} \underbrace{b}_{\beta_1} \underbrace{\lambda}_{x'_1}, \underbrace{\lambda}_{y_2} \underbrace{c}_{\alpha_2} \underbrace{a}_{\beta_2} \underbrace{a^{m-1}}_{y'_2}) \rightarrow_r (\underbrace{a^{n-1}}_{x_1} \underbrace{a}_{\alpha_1} \underbrace{a}_{\beta_2} \underbrace{a^{m-1}}_{y'_2}, \underbrace{c}_{y_2\alpha_2} \underbrace{b}_{\beta_1x'_1}) \\ &= (a^{n+m}, cb) \\ &= (z, w) \end{aligned}$$

可以将剪接操作当作基本工具来建造一种生成机制, 称为剪接系统. 给定一个串的集合 (公理集) 和剪接规则的集合, 所生成的语言由如下方法得到的串组成: 从公理集开始, 在公理集和已经获得的串上重复使用剪接规则.

如果使用集合的传统含义, 我们实际上已经默认, 剪接 x 和 y 得到 z 和 w 后, 我们仍可以将 x 和 y 当作剪接项, 与此相似, 对新生成的 z 和 w 也没有数量上的限制. 但更合理的假定应当是, 一些串仅可使用有限次. 在数学用语上, 不用集合来表示剪接项, 而用多重集, 即在每个时刻都应当记录每个串可用的个数.

依据 [14] 的方式, 如果 N 是自然数集, Σ^* 的一个多重集是映射 $M: \Sigma^* \rightarrow \mathbb{N} \cup \infty$, 对于词 $w \in \Sigma^*$, $M(w)$ 表示 w 的个数, 如果 $M(w) = \infty$, 则意味着 w 的个数不受限制. 集合 $\text{Supp}(M) = \{w \in \Sigma^* | M(w) \neq 0\}$ 称为 M 的支撑集. 现在介绍剪接系统.

定义 1. 剪接系统是一个四元组 $r = (\Sigma, T, A, R)$, 其中 Σ 是一个字符集, $T \subseteq \Sigma$ 是终结字符集, A 是 Σ^* 上的多重集, $R \subseteq \Sigma^* \# \Sigma^* \$ \Sigma^* \# \Sigma^*$ 是剪接规则的集合.

剪接系统 γ 定义了一个 Σ^* 上多重集的集合上的二元关系 $\Rightarrow_\gamma$. 对于多重集 M 和 $M', M \Rightarrow_\gamma M'$ 当且仅当存在 $x, y \in \text{Supp}(M), z, w \in \text{Supp}(M')$, 使得:

- (i) 若 $x \neq y$ 则 $M(x) \geq 1, M(y) \geq 1$ (相应地, 若 $x = y$ 则 $M(x) \geq 2$)
- (ii) 根据剪接规则 $\gamma \in R$ 有 $(x, y) \rightarrow_\gamma (z, w)$
- (iii) 若 $x \neq y$ 则 $M'(x) = M(x) - 1, M'(y) = M(y) - 1$ (相应地, 若 $x = y$ 则 $M'(x) = M(x) - 2$)
- (iv) 若 $z \neq w$ 则 $M'(z) = M(z) + 1, M'(w) = M(w) + 1$ (相应地, 若 $z = w$ 则 $M'(z) = M(z) + 2$)

直观的讲, 给定一个串的“集合”, 其中的串 (可能无限个) 都有一定的重复量, 通过剪接其中的两个串可生成下一个“集合”. 剪接之后, 剪接项被消费 (其重复数减 1), 剪接结果加入到“集合”中 (其重复数增 1).

由剪接系统 γ 生成的语言定义为

$$L(\gamma) = \{w \in T^* | A \Rightarrow_\gamma^* M \text{ 且 } w \in \text{Supp}(M)\},$$

其中 A 是公理集, $\Rightarrow_\gamma^*$ 是 $\Rightarrow_\gamma$ 的自反传递闭包.

给定两个 0 型语言的集合 F_1 和 F_2 . 记

$$H(F_1, F_2) = \{L(\gamma) | \gamma = (\Sigma, T, A, R), A \in F_1, R \in F_2\}.$$

(记号 $H(F_1, F_2)$ 来自 Tom Head, 是他首先给出了剪接的定义).

例如, $H(\text{FIN}, \text{REG})$ 表示由剪接系统生成的语言集, 其中公理集是有限的, 而规则集是一个正则语言, $A \in F_1, R \in F_2$ 的剪接系统 $\gamma = (\Sigma, T, A, R)$ 称为属于型 (F_1, F_2) .

人们对剪接系统已经进行了广泛的研究, 这些研究包括各种类型剪接系统的生成能力决定性问题, 主模型的变种 (带有允许 / 禁止条件的剪接系统, 线性或环形剪接系统, 图形剪接系统, 分布式剪接系统). 至于细节, 读者可参阅 [27] 和 [39] 及其参考文献.

DNA 计算机的存在性

定义了 DNA 计算的数学模型后, 利用这种模型, 我们现在来回答 DNA 计算的完备性与通用性问题.

我们将首先表明剪接系统是计算完备的, 这里完备性是指每个算法 (有效过程) 可以用剪接系统来实现. 但这显然不是计算完备性的数学定义, 对于完善的定义, 算法 (有效过程) 的直观含义一定可以用形式含义来代替.

1936 年, [48] 介绍的通用计算模型——图灵机被广为接受. 计算机科学领域中具有广泛影响的 Church-Turing 论断, 称任何实际的计算机的能力都不会超过图灵机. Church-Turing 论断得到了人们的普遍承认, 其原因是对有效过程的各种形式化方法, 最后都证明与图灵机是等价的. 这些形式化方法中包括马尔科夫规范算法, Post 规范系统, λ 型语言以及“可计算”函数.

证明剪接系统是计算完备的, 等于要证明任何可计算函数可以用剪接系统来实现. 下面将解释“可计算函数”的含义 (参见 [49]).

将笛卡尔幂集 N^n 的一个子集映射到 N . 这样的函数称为偏函数, 这里 $n \geq 1, N$ 是自然数集. 如果该函数的定义域等于 N^n , 则称之为全函数. 全函数的例子有 (对 n 的不同取值):

零函数: $Z(x_0) = 0$, 对任意 $x_0 \in N$.

后继函数: $S(x_0) = x_0 + 1$, 对任意 $x_0 \in N$.

投影函数: 对任意 $i, n, x_i \in N, 0 \leq i \leq n, u_i^{n+1}(x_0, x_1, \dots, x_n) = x_i$.

偏递归函数集可定义为包含特定的基本功能并在特定操作下封闭的最小集.

一个 $(n+1)$ 元的函数 f 可以用函数 f 和 g 中的递归模式来定义. 如果

$$f(0, x_1, \dots, x_n) = g(x_1, \dots, x_n),$$

$$f(k+1, x_1, \dots, x_n) = h(f(k, x_1, \dots, x_n), k, x_1, \dots, x_n).$$

合成操作将函数 $h, g_0, \dots, g_k$ 与如下定义的函数 f 联系起来:

$$f(x_0, x_1, \dots, x_n) = h(g_0(x_0, \dots, x_n), \dots, g_k(x_0, \dots, x_n)),$$

其中对参数 $(x_0, \dots, x_n)$, 每个 $g_i (0 \leq i \leq k)$ 及相应 h 的值已明确定义.

称 f 是用 g 上的最小化操作定义的, 如果

$$f(x_0, \dots, x_n) = \begin{cases} (\mu y)[g(y, x_0, \dots, x_n) = 0], & \text{若存在这样的 } y \\ \text{未定义}, & \text{否则.} \end{cases}$$

对于给定的 $(x_0, \dots, x_n)$, 当 $g(y, x_0, \dots, x_n) = 0$ 时, 其值是最小的 y 值; 如果没有满足上述条件的 y 值, 则 f 的值未定义.

一个函数 f 是偏递归的, 如果

- (i) 它是零函数, 后继函数或投影函数;
- (ii) 它是由偏递归函数合成的;
- (iii) 它是由来自偏递归函数中的递归模式定义的, 或者
- (iv) 它是由既为全函数又为偏递归函数上的最小化操作定义的.

人们已经证明一个函数 f 是偏递归的当且仅当存在一个图灵机可以计算 f 的值. 另一方面, 根据 Church-Turing 论断, 任何可被有效计算的事情都可以被图灵机计算. 据此, 偏递归函数也可称为 (有效) 可计算函数.

与可计算函数等价的语言是 0 型语言, 即由不在其生成规则上施加任何限制的语法所生成的语言. 人们可以证明一个语言属于 0 型语言, 当且仅当其特征函数是可计算的. 语言 $L \in \Sigma^*$ 的特征函数是指 Σ^* 的函数 ϕ : 若 $w \in L$ 则 $\phi(w) = 1$, 否则 ϕ 未定义.

回想一下, 0 型语言的集合记作 L_0 , 剪接系统计算完备性的结论可以表述如下:

定理 1. $L_0 = H(\text{FIN}, \text{FIN})$.

直观的讲, 本定理断定每一个 0 型语言都可以由带有有限条公理和有限条规则的剪接系统生成. 给定一个 0 型语法 G 生成的语言 $L(G)$, $L_0 \subseteq H(\text{FIN}, \text{FIN})$ 的证明由两步组成: (a) 构造一个剪接系统 $\gamma \in H(\text{FIN}, \text{FIN})$, 它模拟了语法 G 的生成规则; (b) 证明所构造的剪接系统可以生成给定的 0 型语言, 即 $L(\gamma) = L(G)$. 至于 $H(\text{FIN}, \text{FIN}) \subseteq L_0$ 则可以直接证明, 也可以引用 Church-Turing 论断. (定理 1 的证明技巧来自 [14], 并首次在 [38] 中给出, 至于细节读者可参考 [16]).

从可计算函数的角度讲, 定理 1 断言任何可计算函数都可由剪接系统来实现. 同样, 它断言任何图灵机可计算的事情也可以由这种 DNA 计算模型来计算. 这就回答了哪种算法 (有效过程, 可计算函数) 可由 DNA 计算设备来模拟的问题, 答案是: 全部.

定理 1 表明任何程序 (可计算函数) 均可由剪接系统来模拟, 但它并未对是否存在基于剪接的可编程计算机这一问题给出任何回答. 为解决这个问题, 有必要找到一个通用剪接系统. 通用剪接系统带有剪接系统的全部要素, 但给定任一剪接系统 γ , 可以将 γ 的代码写入到通用系统的公理集中, 从而使得通用系统能够完成 γ 的功能. 将之形式化:

定义 2 给定一个字符集 T 和两个 0 型语言集合 F_1, F_2 , 称结构 $\gamma_u = (\Sigma_u, T, A_u, R_u)$ 是属于型 (F_1, F_2) 的通用剪接系统. 如果对属于型 (F'_1, F'_2) 的每个 $\gamma = (\Sigma, T, A, R)$, 都存在 A_r 使得 $A_u \cup A_r \in F_1$ 且 $L(r) = L(r'_u)$, 这里 Σ_u 是字符集, $T \subseteq \Sigma_u, A_u \in F_1, R_u \subseteq \Sigma^* \# \Sigma^* \# \Sigma^* \# \Sigma^*, R_u \in F_2, r'_u = (\Sigma_u, T, A_u \cup A, R_u)$.

值得注意的是通用系统的类型是固定的, 但通用系统可以模拟属于任何类型 (F'_1, F'_2) 的系统, 其中 F'_1, F'_2 是 0 型语言的集合. 根据定义 2, 我们现在可以给出有关通用性的主要结论:

定理 2. 对每个给定的字符集 T , 都存在一个剪接系统, 其公理集和规则集都是有限的, 而且对于以 T 为终结字符集的一类系统是通用的.

该定理的证明基于定理 1 和通用的 0 型语言 (或通用图灵机) 的存在性. 至于证明的细节, 读者可参看 [16]. 另一个证明是基于这样的事实, 即由 Post 系统生成的语言也可以由剪接系统生成, 该证明见于 [15].

从 DNA 计算的角度对定理 2 的解释是, 理论上讲, 存在一个基于剪接操作的通用可编程 DNA 计算机. 程序由往通用计算机公理集中添加的字符串组成. 程序会有多个, 而可利用的公理集合有无穷多个. 计算机中“固定”的公理可解释为为了运行程序而不断提供给 DNA 计算机的能量. 这些计算机使用的生物操作只有合成, 剪接 (切割 / 连接) 和抽取 (在数学上等于结果和 T^* 的交, 其中 T 是终结字符集). 针对剪接系统, 我们可以得出这样的结论, 即定理 2 为可编程 DNA 计算机的存在性问题提供了肯定的答案.

人们从剪接系统的几个变种也获得了与定理 1 和 2 相似的结论. 例如, 如果作为多

重集的公理集被一个控制条件代替,相似的结论也成立。这个控制条件是:仅当某些叫做允许上下文的串在剪接中出现,一个剪接规则才能发挥作用(参见[16])。

用生物操作的组合来建立 DNA 计算模型,并用这种模型来模拟图灵机的其它方法见于文献[2,9,11,40,42,43,44,47,52]。从乐观的角度看,我们可以认为这些工作与 30 年代寻求计算模型的工作相似,那些工作为电子计算机设计打下了基础。

回到实处:实现技术

众所周知,从数学的理想世界到分子生物学实验世界的转换过程中充满了挑战。确实,追逐梦中的幻影并不是件轻松的事。这一点,毕生从事 DNA 计算模型研究的科学家们可以证实。尽管已经取得了一些进展,但开发实用 DNA 计算机的主要障碍还未克服。这些障碍大致可以分为两类[2]:

- **实践上**,问题主要在于如何处理大规模系统和时付误差;
- **理论上**,是如何使 DNA 计算机集成多种功能及其对多种可计算难题的有效处理能力。

本节中我们将考察这两类问题,重点放在已有模型的实际技术上。我们将指出实现本文中每个生物操作的潜在困难。这些问题看起来没有不可解决的,但认识问题只是解决问题的第一步。

我们首先从合成操作开始。合成 DNA 链时有时会导致一个链跟它自己融合在一起,形成发夹式在结构。即便看起来很直接的混合操作有时也会带来麻烦:如果处理得不小心,注射和混合中轻微的振荡也会使 DNA 断裂。还要考虑是有些 DNA 会粘在试管壁、抽筒或吸量管的尖上,从而就在计算中丢失了。

由于融合所需的热动力参数与序列有关,杂交过程也需要认真监测,这是因为在 DNA 的反应条件下,两个寡核苷酸即使基对间不完全匹配也可以杂交。可以将 DNA 寡核苷酸匹配时互补的基对数称为杂交强度,它依赖于反应条件(盐浓度,温度, A 和 T 对 G 和 C 的百分比,反应时间),并随温度升高而增长。提高杂交强度的一种方法是问题的输入信息选择好的编码[13]。[7]中为避免自融合和错配问题提供了另一种方案:在编码时选择特殊的序列作为空格符将信息位分开。

利用 PCR 放大建立在如下的假设上:如果有比模板多的引物,就可以避免模板间的交叉。[26]中已经指出,这个假设未必成立。实际上,实验证据表明,确实会生成一些复杂的结构,如折叠的 DNA、多链复合、不确定的连结物等。这可能进一步影响利用凝胶电泳法来分离链的准确性。确实,有了这些复杂的结构,链的移动性不仅依赖于其长度,还和 DNA 结构(形状)有关。作为一种可能的解决方案,[28]推荐使用变性或单链凝胶法来分析。而且,通过保持低浓度,异源螺旋(带有错配的双链)结构以及模板间的交叉会减到最小。

按长度分离链以及抽取包含给定模式的链,这两种操作效率可能会很低,这会为大规模的试管方法带来困难。[33]中提供的方法是,开始时将寡核苷酸粘在(玻璃,硅,金或小珠的)表面上,然后为了获得所要求的溶液,将它们参与到诸如标识,去标,破坏

等生物操作中,这个方法可以极大地减少在利用亲和净化法抽取时 DNA 分子的丢失。其缺点是依赖于标记和去标操作,这反过来又假定可以确定和辨别单基的错配。尽管使用 15- 聚物序列时已经证明这些过程是可靠的,但在处理更长或更短多核苷酸链时却变得很困难。另一个困难是计算的规模受到表面方法的二维性的限制:人们无法获得太高的信息存储密度。

抽取 含有给定模式的链时效果也不是 100% 地好,有时会留下不含有这些模式的链。尽管在抽取量很小时误差率是合理的,但当抽取量是成千上万时,即使有效率是 95% 也会出现问题。能够得到仅包含指定链而无杂质的溶液的概率是很低的。作为另一种可能的方案, [5] 中提供了删除操作来取代抽取操作。删除是一个复合操作,它从链的集合中清除其中至少包含指定序列一次的那些链。该操作首先标记所有包含给定序列的链,然后将这些链破坏掉。本方法的优点是用于删除操作的限制性酶的误差率比抽取操作低得多。它的一个缺点是,尽管开始时试管中会有每个链的多个备份,但经过多次删除操作后,材料的量会低得无法接受。但可以阶段性的利用 PCR 放大来避免这个问题。

利用限制性外切酶来切割(分裂)DNA 链也是指利用这种酶来消化 DNA。但这个过程中有时会产生不完全消化的产品。人们必须为所用酶的有效性而测试所有流程,还常常有必要找到清除来消化材料的方法。与此相似,连接的精确性虽然很高,但并不完美。一种连接酶可能会将错误的分子连到粘性末端上,如果它与应当连结的分子非常相似。

检测和序列化操作通常需要酶化反应和凝胶电泳过程,但这两种过程即费时又昂贵。克服这种缺陷的一个方案是利用杂交来完成序列化,这种技术可在单步自动过程中读取输出。在本方法中,目标链与在平面上合成的所有寡核苷酸杂交,而后从得到的杂交数据中重构目标链。为避免自融合引起的误差,该方法推荐了一个受限模板字符集,它仅含四个基中的两个。这样试管中内容不再会发生分子内反应,而分子间反应则不受影响。

除了生物操作的准确性,实现 DNA 计算的另一个难题是问题的规模会影响反应物的浓度,从而反过来影响最终反应产品的生产率和质量。[30] 分析了 Aldeman 的实验,结果表明即使针对稀疏的有向图,溶液的浓度也会成指数级地降低,而且对于那些不比 Aldeman 所解决的大太多的问题,这种浓度的丢失也需引起足够的重视。对于使容量缩小的那些 DNA 算法,预防误差的一个方案似乎是重复利用 PCR 放大中间产品。但这并不总能解决问题,因为并非所有算法都使容量缩小。事实上,正如 [21] 所指出的,规模化 DNA 计算的一个障碍是 DNA 的数量。在某些情况下 [3], 为将误差率降到所要求的程度,需要的 DNA 大约是地球上所有的 23 倍——这种设计是不可接受的! 为减少 DNA 的数量,可能需要多种算法变换。

确实,我们不应该对生物技术上的困难喋喋不休了,似乎推进 DNA 计算研究的唯一途径是等待生物技术的进展。我们必须转向另一种方案:通过新的程序设计技术,采用概率算法及修改数学问题传统的解决方案来提高 DNA 计算的精确性。

算法变换已被用在解决分类问题的 DNA 算法中了。该分类的任务是,给定一个候选方案库,针对一个问题将库中的方案分为两类,一类可以解决该问题,而另一类不能。

在这种情况下(包括 Adleman 的实验),主要误差是假阳性,假阴性和链丢失。解决这类问题的一种可能的方案是通过智能的时间-空间折衷来修改算法。

[2] 中介绍了一种称为重复的基本变换方法,该方法使用一个减速因子 M ,对给定算法 A ,可得到新算法 A' ,后者误差率要比前者小:

- 重复 M 次:

在输入 I 上执行 A ,生成试管 Y 和 N ;丢弃 N ,将 Y 命名为 I 。

- 将 I 作为“是”返回,而空试管作为“否”返回。

如果已知原算法 A 会非常可靠地将好的序列置于“是”试管中(即低的假阴性),而将坏的序列置于“否”试管中(即高的假阳性),那么这种变换方法是很有价值的。如果已知算法有高的假阴性和低的假阳性,可以使用上述方法的一个变种。本方法及其相似的方法用于 [3] 和 [4],其中的模型利用粘着剂(短互补序列)来标记“on”位,来标记的位称为“off”。粘着剂模型的优点是它不依赖酶这种短生命的材料,也不使用 PCR 那样的耗能过程。与此相对的是,有人可能更乐意用 PCR 来解决误差不太严重的问题。

至此我们已经看到,在 DNA 计算研究的理论方面,几个主要障碍都已经克服,这意味着分子计算的实际应用在未来是可行的。本节中我们讨论了实现技术及相关的误差率问题,表明几乎在各个阶段都有实际工程上的挑战。然而要知道,主动检测和调整反应物浓度的问题,以及误差容错的问题,都已被大自然中的生物系统涉及:细胞要控制各种成分的浓度,要安排稀有分子的反应,要处理其自身活动中额外的付产品。这些问题在生物体内已成功的解决了,就不会在生物体外解决不了。作为朝着这个方向的一步,[30] 提出了一种建立在分割膜和用膜过滤化学物质活动系统的基础上的机制。[4] 和 [50] 也提出了如何将电子计算机的设计原理用于分子计算机。

描述了实践上的障碍后,让我们回到建造分子计算机的理论问题上。争议最大的问题之一是,对于各种实际应用,分子计算机是否会超过并最终取代电子计算机;或者是否有更适于用 DNA 计算机来解决的问题。确实,单个试管中的 DNA 溶液可能表示 10^{18} 字节的信息,源于此潜在的并行机制使得 DNA 成为吸引人的计算介质。同样吸引人的是每千克 10^{23} 字节的存贮密度,这使人类的基因组也相形见绌:仅 10^9 字节 [50]。这些特征会使人猜想,无论什么应用, DNA 计算机都会比电子计算机更有效,更快。可能会这样吧,但下面的对比提醒我们,即便在理论上,也不要期望过高。

在现实生活中,需要快速,有效,并行的处理大量数据,这对,人的能耐要超过最时髦的计算机。例如,我们每天要遇到很多人,我们脑子里的计算机会在霎那间处理由传感器送来的数据,比如对方的形象,个头,肤色,声音,气味,手势,动作以及表情等。信息以光速处理后结果就出来了:是朋友还是陌生人,该不该微笑,该不该接触。与之比起来,最先进的机器人也要逊色万分,因为它在为最蠢笨地挪动而处理并行的传感器输入时还困难重重,更不用提社会化了。但问题的另一面是,如果要将一百万个 50 位的数字加起来,电子计算机要远胜过人的大脑。

这样一来我们或许不能期望用 DNA 计算机完全取代电子计算机。具有高度并行性算法的问题可能成为 DNA 计算机的领地,而其算法生来是顺序的问题或许仍是电子计算机的专利。事实上,寻找一个“应用杀手”——即 DNA 方案明显优于电子方案的应

用——是这个领域的主要研究方向之一。同时，人们正致力于检测生物操作的可实现性，希望最终找到一个“原语”集作为分子高级编程语言的基础。最后，但仍很重要是的，人们期望那些探求生物分子计算局限性的实验性工作可以为我们带来对细胞组织信息处理能力的更深的理解。

生物数学的元思考 (meta-thoughts)

我们已经看到生物操作与常规数学操作之间有很大的不同。事实上，比起量上的区别来，虚拟 DNA 计算机与电子计算机间质上的区别更引人注目。

DNA 计算是考察计算问题的一种全新的方式。或许这正是大自然做数学的方法：不是用加和减，而是用切割和粘贴，用插入和删除。正如用十进制计数是因为我们有十个手指那样，或许我们目前计算中的基本功能仅因为人类历史使然。正如人们已经采用其它进制计数一样，或许现在是考虑其它的计算方式的时候了。

生物机体中所发生的现象 (DNA 链的复制，切割，粘贴) 都是带有面具的计算，这一事实表明生命本身就是由一系列复杂的计算组成的。由于生命是最复杂的自然现象之一，我们可以猜想整个宇宙也是由计算组成的。事物之形形色色反映的仅仅是计算的复杂程度的多样性，其量上的差别反应了计算的巨大加速空间，从混沌到无机物，从无机到有机，再到意识和思维，或许世界的整个进化过程就是计算复杂性不断增长的过程。

当然，上面谈到的仅仅是假设，要知道，我们连现代人是“现代人类”还是“人类计算物”的谜还没有解开。但正缘于此，DNA 计算才如此迷人。它不仅使计算更快更有效，更开发了人们的想象，揭发了更深奥的哲学问题，让我看到了从未有过的希望。

对数学家而言，DNA 计算揭示了数学可能是万物的基础。确实，人们已经证明，对于物理，化学，音乐，虚拟艺术 [23]，语言等等学科来说，数学是其固有的一部分。DNA 计算的发现也表明数学居然也存在于生物学的根基上！这使人猜测数学会是所有已知和可能世界的核心。

或许，柏拉图真是对的：真，善，美同而为一。或许，物质真的仅仅是意识的外在表现，而意识是永恒的，不生不灭。DNA 计算宣称数学处于生命的核心，这就使我们把柏拉图的哲学推进了一步：存在于短暂物质世界的永恒“意识”就是数学意识。

如果这是真的，而且存在的本质就是数学的客观世界，那么我们责无旁贷，为之不懈努力。

致谢： 本文获益于同下述人士的讨论：Leonard Aldeman, Chandler Davis, Mark. Gijzen, Greg Gloor, Tom Head, Jarkko Kari, Gheorghe Paun, Clive Reis, Arto Salomaa, Gabriel Thierrin, Gary Walsh. 在此对他们特表谢意。还要特别感谢 Luiz Merkle，是他提供了分子过程的图片。这里恕我无法一一指出影响本文的文献，因此，引用 Seneca 的名言，“无论谁的良言，皆为人所有 [46].”我这里要感谢文献中的所有作者。

参考文献 (略)

(下转 273 页)

- [C3] H. Cartan, Notions d'algèbre différentielle; application aux groupes de lie et aux variétés où opère un groupe de Lie, *Colloque de Topologie*, C. B. R. M., Bruxelles, 1950, p. 15–27; *Collected Papers III*, pp. 1255–67.
- [C4] S. S. Chern, Topics in differential geometry, Notes, Institute for Advanced Study, 1951.
- [GHV] W. Greub, S. Halperin, and R. Vanstone, *Connections, Curvature and Cohomology*, Vol. 3, Pure Appl. Math. Vol. 47, Academic Press, 1976.
- [S1] A. Selberg, On discontinuous groups in higher-dimensional symmetric spaces, *Contributions to Function Theory*, Internat. Colloquium TIFR, Bombay, 1960, pp. 147–164.
- [S2] J. -P. Serre, Faisceaux algébriques cohérents, *Ann. Math.* 61 (1955), 197–278; *Collected Papers I*, pp. 310–91.
- [W1] A. Weil, *Foundations of Algebraic Geometry*, Colloquium Publ., Vol. XXIX, Amer. Math. Soc., New York, NY, 1946; Second edition, Providence, RI, 1962.
- [W2] ———, Fibre Spaces in algebraic geometry (Notes by A. Wallace), Notes, University of Chicago, 1952.
- [W3] ———, *Collected Papers*, 3 volumes, Springer-Verlag, New York, 1979.

(胥鸣伟 译 余建明 校)

(上接 291 页)

作者: LiLa Kari 在罗马尼亚的 Bucharest 大学获得数学 / 计算机科学的硕士学位。攻读其博士时师从 Salomaa. 博士论文是关于形式语言的插入和删除问题, 该论文获得在芬兰举办的 1991 年度 Levanlinna 奖的最佳数学博士论文奖。在芬兰的 Turku 大学工作两年之后, 她来到了加拿大的 Western Ontario 大学, 并一直工作至今。最近, 她离开数学系转入计算机科学系。除了进行 DNA 计算的研究外, 她的研究兴趣是计算理论和离散数学。

(谭红星 译 陆维明 校)